

AlphaRead — Project Technical Documentation

Full-Stack Financial GenAI RAG Application (FastAPI + React + Llama-3)

1. Executive Summary

AlphaRead is a production-grade, full-stack Retrieval-Augmented Generation (RAG) web application engineered for querying, analyzing, and citing financial statements, annual reports, and US SEC 10-K filings in real time. It combines a FastAPI Python backend with a humanized React frontend (Vite + Tailwind CSS v4), embedding financial documents via HuggingFace sentence transformers, storing dense vectors in memory, and invoking Groq's high-speed **Llama-3 LLM** to deliver fact-grounded analytical reports with interactive, verifiable Source Citations.

2. Live Cloud Deployment Specifications

Component	Platform	Live Public URL
Frontend Web UI	Vercel (Global Edge)	https://alpha-read-financial.vercel.app
Backend REST API	Render (Python 3)	https://alpharead-backend.onrender.com
GitHub Repository	GitHub (Public)	https://github.com/nagul-2005/AlphaRead-Financial-RAG

3. Core Application Features

- **Dual Ingestion Pipeline:** Drag-and-drop PDF statement uploader alongside a 1-click SEC EDGAR 10-K ticker fetcher (AAPL, NVDA, MSFT, TSLA, AMZN, GOOGL, META, etc.).
- **Dynamic SEC Section Selection:** Allows users to customize 10-K section extraction prior to ingestion (Item 1 Business, Item 1A Risk Factors, Item 7 MD&A, Item 8 Financial Statements).
- **Granular Document Deletion:** Individual trash icon (DELETE /documents/src) enabling deletion of single document vectors without wiping the entire vector memory.
- **High-Speed Vector Engine:** FastEmbed ONNX runtime sentence embeddings (BAAI/bge-small-en-v1.5 / all-MiniLM-L6-v2) operating under 60MB RAM footprint for low-latency retrieval.
- **Llama-3 Generative Reasoning:** Sub-second Groq API integration (llama-3.3-70b-versatile) producing structured financial summaries with inline source attribution.
- **Interactive Source Citations:** Every AI response features a collapsible drawer showing exact document names, section/page numbers, relevance match scores (%), and verbatim text snippets.

4. Technical Stack & Architecture

Layer	Technology	Description / Purpose
Frontend UI	React 18, Vite, Tailwind v4	Humanized light-theme dashboard, responsive 2-column layout.
Backend Web Server	FastAPI, Uvicorn, Python 3.13	Asynchronous REST API with guaranteed CORS HTTP middleware.

Document Parsers	pypdf, pdfplumber, edgartools	Parses uploaded PDF text & SEC EDGAR 10-K sections.
Chunking & Embeddings	LangChain, FastEmbed ONNX	RecursiveCharacterSplitter (1000/200) + fast ONNX dense vectors.
Vector Store	ChromaDB / Local Vector Engine	Cosine similarity search with persistent disk storage.
LLM Integration	Groq API (Llama-3.3)	Generates natural language financial answers with citations.

5. API Endpoints Reference

HTTP Method	Endpoint Path	Function & Return Payload
GET	/health	Checks backend status & Groq API configuration state.
POST	/upload	Ingests PDF statement, returns pages & chunks created.
POST	/ingest-sec	Fetches ticker 10-K sections, returns vector chunk stats.
POST	/chat	Vectorizes query, retrieves top 3 chunks, returns answer & citations.
GET	/documents	Lists all ingested documents, doc_types, and vector counts.
DELETE	/documents/{src}	Deletes vector embeddings for a specific source document.
DELETE	/clear	Resets vector store index completely.